

yt-chat

Ask questions about any YouTube video
and get answers with clickable timestamps

A Complete Project Walkthrough

Python · RAG · Command Line

What it is	A terminal app that reads a YouTube video's subtitles and answers your questions about it.
The rule it follows	It only uses what the speaker actually said. If the video does not cover it, it says so instead of guessing.
What you get back	A short answer, plus timestamps you can click to jump to that exact moment in the video.
Built with	Python, SQLite, FAISS, Sentence-Transformers, Typer, and a free LLM API (Groq, Gemini, OpenRouter) or a local model.
Tested on	3 videos totalling 6 hours 45 minutes, with 35 hand-written questions.
Test suite	195 tests, run in about 2 seconds, with no internet needed.

1. What This Project Does

Imagine watching a three-hour technical talk and wanting to know one specific thing. Scrubbing the timeline is slow. Searching the web gives answers from somewhere else, which may not match what this particular speaker said.

yt-chat solves that. You give it a YouTube link. It reads the video's subtitles, then lets you ask questions in plain English. Every answer comes with the timestamp where the speaker said it, so you can verify it yourself in one click.

The one promise this project makes

The assistant never invents an answer. If the information is not in the video, it replies: *"I cannot find this information in the video."* Three separate safety checks enforce this. Measured across 11 questions the videos genuinely do not answer, it refused **every single one**.

How it works, step by step

Six stages. The first time you use a video it runs all six. Every time after that it loads saved results instantly.

Stage	What happens
1. Get the words	Downloads the video's subtitles, including the exact time each line was spoken.
2. Clean them up	Removes noise like [Music] and speaker labels. Auto-subtitles repeat the previous line's ending, so those duplicates are stripped too.
3. Cut into pieces	Splits the transcript into passages of about 700 characters, roughly 36 seconds of speech. Each piece keeps its start and end time.
4. Turn into numbers	Converts each piece into a list of numbers that captures its meaning, so similar ideas end up close together.
5. Find the right pieces	When you ask something, it finds the handful of passages most likely to contain the answer.
6. Write the answer	Sends only those passages to an AI model with strict instructions: use nothing else, cite your sources, and refuse if the answer is not there.

The clever part: timestamps never get lost

Most similar projects glue the subtitles into one block of text and then guess which timestamp a passage came from. This project instead records exactly which characters belong to which moment, so a passage starting halfway through a subtitle line still gets an accurate time. That is what makes the citations trustworthy rather than approximate.

2. Features

Everything below is built and covered by tests.

Reading the video

- Accepts every YouTube link format: normal links, short `youtu.be` links, Shorts, embeds, live links, or just the bare video ID.
- Prefers human-written subtitles; falls back to auto-generated, then translated ones.
- Adapts to subtitle quality. Human subtitles have punctuation, so it splits on sentences. Auto-subtitles often have none, so it switches to fixed-size word windows.
- Fetches title, channel and duration where available, but never lets a missing title stop you asking questions.

Answering questions

- Answers come only from the video, in one to four sentences, in the speaker's own terms.
- **Three independent refusal checks.** If nothing relevant was found it refuses without calling the AI at all, saving an API call. The AI is also instructed to signal when the passages do not answer the question. Finally, any answer citing a source that does not exist is thrown away.
- Every sentence is tagged with a source number, and every source becomes a clickable timestamp link.
- Follow-up questions work. Ask "What is a jailbreak?" then "Why does that work?" and it rewrites the second into a full question before searching.

Searching

- **Meaning-based search** finds the right passage even when you use different words.
- **Keyword search** matches exact names, numbers and jargon.
- **Hybrid search** combines both.
- **Reranking** (optional) takes the top 30 results and rescores them with a slower, more accurate model.
- You can switch between all four at any time, even mid-conversation.

Speed, storage and measurement

- Each video is processed once, then stored locally, so the second run is instant.
- The cache is precise about what to redo. Change the passage size and it re-cuts but does not re-download. Change the AI model and it keeps the passages.
- A built-in evaluation system scores how well it finds the right passages and how trustworthy the answers are.
- Correct answers are defined by *time ranges*, not passage numbers, so the test set stays valid even after changing how the transcript is cut up.
- It measures something most projects skip: whether the timestamp actually points at the right moment, not just whether the answer was right.
- It can tune its own refusal threshold without spending a single API call.

Choice of AI model (all free)

Option	Notes
groq	Recommended. Free, very fast, runs Llama 3.3 70B. Free key required.
gemini	Google AI Studio free tier. Free key required.
openrouter	Several free community models. Free key required.
ollama	Runs entirely on your own computer. No key, no limits, works offline.

3. Commands

Every command starts with **yt-chat**. Run them from the project folder with the virtual environment active.

Starting a conversation

```
yt-chat https://youtu.be/zjkBMFhNj_g
```

Or paste just the video ID:

```
yt-chat zjkBMFhNj_g
```

Useful options:

Option	What it does
<code>--retriever dense</code>	Search method: dense, sparse or hybrid.
<code>--top-k 8</code>	How many passages to look at. More context, slower and pricier.
<code>--llm groq</code>	Which AI service to use for this run.
<code>--force</code>	Ignore the saved cache and reprocess from scratch.
<code>--quiet</code>	Hide progress messages.

Commands inside the chat

Once the session is open, just type your question. Or use these:

Command	What it does
<code>/ask what is the LLM OS?</code>	Asks a question. Typing the question directly works too.
<code>/sources</code>	Shows sources for the last answer, with quotes.
<code>/history</code>	Shows the whole conversation so far.
<code>/change-retriever sparse</code>	Switches search method without restarting.
<code>/compare what is a jailbreak</code>	Runs all three search methods side by side.
<code>/video</code>	Title, length, number of passages, models in use.
<code>/debug</code>	Explains the last answer: passages used, their scores, and which safety check refused it if it refused.
<code>/clear</code>	Forgets the conversation so far.
<code>/help</code>	Lists these commands.
<code>/exit</code>	Quits.

Asking a single question

For scripts, or when you want one answer without a chat session.

```
yt-chat ask https://youtu.be/zjkBMFhNj_g -Q "What is the LLM OS?"
```

It exits with code 0 if it answered and 1 if it refused, so it can be used inside other scripts.

Managing saved data

```
yt-chat cache stats
```

```
yt-chat cache clear --video https://youtu.be/zjkBMFhNj_g
```

The second deletes everything saved for one video. It deliberately refuses to wipe the whole cache in one command.

Measuring quality

Finds the best refusal threshold. Costs nothing, because it makes no AI calls:

```
yt-chat eval calibrate benchmarks/karpathy_llm_intro.yaml -r dense
```

Compares all search methods. Add --no-judge to skip AI scoring and make it free:

```
yt-chat eval run benchmarks/karpathy_llm_intro.yaml --no-judge
```

Runs the full evaluation, resuming wherever the daily free-tier allowance ran out:

```
python scripts/run_eval_queue.py
```

Tests how passage size affects quality. Free to run:

```
python scripts/sweep_chunk_size.py
```

Changing settings

Every setting can be changed through the `.env` file, without touching code. This is how experiments are run.

Setting	Meaning
<code>YTCHAT_LLM_PROVIDER</code>	Which AI service: groq, gemini, openrouter or ollama.
<code>YTCHAT_RETRIEVER</code>	Default search method.
<code>YTCHAT_MIN_SCORE</code>	How confident it must be before answering. Higher refuses more often.
<code>YTCHAT_TOP_K</code>	How many passages to send to the AI.
<code>YTCHAT_MAX_CHARS</code>	Size of each passage. Changing this re-cuts the transcript.
<code>YTCHAT_EMBEDDING_MODEL</code>	Which model turns text into numbers.
<code>YTCHAT_ENABLE_RERANK</code>	Turn the slower, more accurate reranking on or off.
<code>YTCHAT_ENABLE_QUERY_REWRITING</code>	Turn follow-up question rewriting on or off.

Running the tests

```
pytest -q
```

All 195 tests run in about two seconds and need no internet, no API key, and no model downloads.

4. The Main Files

About 35 source files. These are the ones that matter if you want to understand or change how it works.

The heart of the system

File	What it does
<code>pipeline.py</code>	Start here. Ties everything together: takes a URL and returns a ready-to-query video, skipping any stage already cached. Also holds the conversation manager.
<code>models.py</code>	The core data types. The rule that every piece of text carries a start and end time is written down here.
<code>config.py</code>	Every adjustable setting in one place, readable from the <code>.env</code> file.

Turning a video into searchable pieces

File	What it does
<code>ingestion/url.py</code>	Understands every shape of YouTube link.
<code>ingestion/transcript.py</code>	Downloads subtitles and picks the best available track.
<code>preprocessing/clean.py</code>	Removes noise, speaker labels, and the repeated text that auto-subtitles produce.
<code>preprocessing/timeline.py</code>	The key trick. Maps every character position back to a moment in the video, so any cut point gets an accurate timestamp.
<code>preprocessing/chunking.py</code>	Cuts the transcript into passages, choosing sentences or word windows depending on whether the subtitles have punctuation.

Searching and answering

File	What it does
<code>database/vectorstore.py</code>	The search index. Rebuilds itself if the file goes missing.
<code>retrieval/dense.py</code>	Meaning-based search.
<code>retrieval/sparse.py</code>	Keyword search, written from scratch so it can be tuned.
<code>retrieval/hybrid.py</code>	Combines both.
<code>retrieval/rerank.py</code>	Optional second pass with a slower, more accurate model.
<code>generation/llm.py</code>	Talks to Groq, Gemini, OpenRouter or a local model. Handles rate limits and retries.
<code>generation/prompts.py</code>	The exact instructions given to the AI, kept separate because changing them changes the results.
<code>generation/answerer.py</code>	The three refusal checks live here.

File	What it does
<code>generation/citations.py</code>	Turns source numbers into clickable timestamp links, and throws out invented ones.

Storage, interface and measurement

File	What it does
<code>database/repository.py</code>	All reading and writing to the local database.
<code>cli/app.py</code>	All the terminal commands.
<code>cli/repl.py</code>	The interactive chat session and its slash commands.
<code>evaluation/dataset.py</code>	The test-question format, with answers as time ranges.
<code>evaluation/retrieval_metrics.py</code>	Scores how well the search worked.
<code>evaluation/runner.py</code>	Runs the whole evaluation and the threshold tuning.
<code>scripts/run_eval_queue.py</code>	Resumable evaluation queue for free-tier daily limits.
<code>scripts/sweep_chunk_size.py</code>	The passage-size experiment.

5. What the Measurements Found

Six findings from testing against 35 hand-written questions across three videos. All are real measurements that changed the system.

Finding 1: the confidence threshold was doing nothing

The system refuses when it is not confident enough. That threshold was set by guesswork. Measurement showed the scoring model never produces a score below about 0.45 for ordinary English, even for completely unrelated passages, so the check never once triggered.

Finding 2: combining searches made refusals worse

Hybrid search originally took the higher confidence of its two methods. Keyword search sometimes scores highly on a question the video never answers, simply because a couple of rare words happen to appear. That false confidence overrode the meaning-based score, which had correctly rated it low.

Finding 3: the measuring tool measured the wrong thing

The evaluation looked at 30 passages while the live system only ever looks at five, making reported confidence higher than anything the real system sees. Caught because two tools disagreed with each other.

Finding 4: a scoring formula could exceed its own maximum

One ranking metric returned 1.09 on a scale that stops at 1.0, because several retrieved passages can overlap the same correct answer. Only visible because an impossible number appeared in a results table.

Finding 5: the trustworthiness score can be gamed by refusing

Keyword search scored a perfect 1.00 on faithfulness by refusing half the questions. A refusal claims nothing, so it cannot be unfaithful. A system that refuses everything looks flawless. Faithfulness must always be reported next to a false-refusal rate.

Finding 6: passage size has a sweet spot, not a tradeoff

The usual assumption is that smaller passages give tighter timestamps but find answers less often. Testing four sizes across all three videos showed both measures peak at the same point, around 700 characters or 36 seconds of speech. Timestamp accuracy doubles from 400 to 700 characters and then falls again, because accuracy is highest when a passage is about as long as the answer itself. The default was changed as a result.

Passage size	Length	Answer found	Timestamp accuracy
400 characters	18 seconds	0.75	0.076
700 characters	36 seconds	0.83	0.153
1000 characters	55 seconds	0.79	0.148
1500 characters	85 seconds	0.82	0.129

Why these matter

None of these six could have been found by reading the code or by checking that the program runs. They were only visible once the system was measured against questions with known answers. Two of them were bugs in the measuring code itself, which is the argument for treating an evaluation harness as something that needs testing, not as a trusted authority.

6. Current Status

Area	State
Video processing	Working. Any video that has subtitles.
Question answering	Working. Verified on three videos totalling 6h45m.
Refusing to guess	Working. Refused all 11 unanswerable questions. All three checks covered by tests.
Clickable citations	Working.
Follow-up questions	Working.
Caching	Working. Second run of a video is instant.
Four search methods	Working. Including optional reranking.
Test suite	Working. 195 tests, about 2 seconds, fully offline.
Measuring search quality	Done. Three videos, 35 questions, results published.
Measuring answer quality	Done. All nine runs complete, spread over four days because the free AI tier caps daily usage. 32 of 33 unanswerable questions correctly refused.
Videos without subtitles	Not built. Roughly 15% of YouTube. Would need local speech recognition.

The most revealing single result

Out of 33 questions the videos genuinely do not answer, 32 were refused. The one that slipped through is more instructive than the 32 that did not. A question about convolutional neural networks, asked of the tokenizer video, was answered. The scoring system rated that answer **perfectly faithful** — and it was right to. The model did not invent anything; it found passages it could stretch into an answer and quoted them correctly. Every safety check behaved exactly as designed. The problem was that the question should never have been answered at all.

Only one measure caught it: the score for how much of the retrieved material was actually relevant, which came out at 0.20. The lesson is that a trustworthiness score asks whether a claim is supported, never whether the question should have been answered — so it can never be read on its own.

What is left

- Add speech recognition for videos with no subtitles.
- Publish the repository.

An honest note on the numbers

All results come from 35 questions across three videos by a single speaker. At that size, any difference smaller than about 0.15 is noise rather than a real effect. One finding demonstrates this directly: which search method ranks best *changes* depending on the passage size, so no single method can be declared the winner from this data.

Built with Python 3.12. Requires a free API key from Groq, Google AI Studio or OpenRouter, or a local model through Ollama.
Full technical detail in docs/technical_walkthrough.md; full results in docs/evaluation.md.